Module 13 - Decorators

Table of Contents

- [ ] 1. Why Decorators Matter in Cybersecurity
- [ ] 2. Prerequisites — Functions Are Objects
- [ ] 3. The Wrapper Pattern — Your First Decorator
- [ ] 4. Generic Decorators with `*args` and `**kwargs`
- [ ] 5. `functools.wraps` — Preserving Function Identity
- [ ] 6. Parameterized Decorators
- [ ] 7. Stacking Multiple Decorators
- [ ] 8. Real Security Decorator Patterns
- [ ] 9. Summary and Next Steps

---

1. Why Decorators Matter in Cybersecurity

Decorators solve a problem you will face constantly in security tooling: **you need to add the same behaviour to many different functions** — without copying and pasting code into every single one.

Problem	Decorator solution
Every function in your tool should log when it runs and how long it took	Write one `@audit_log` decorator, apply everywhere
API endpoints should only be called by authenticated users	Write one `@require_auth` decorator
Functions calling external APIs should retry on failure	Write one `@retry` decorator
Expensive lookups (VirusTotal, Shodan) should be cached	Write one `@cache_result` decorator
Functions modifying firewall rules need rate limiting	Write one `@rate_limit` decorator

In real security frameworks (`Flask` , `FastAPI` , `Django`), decorators like `@login_required` , `@permission_required` , and `@csrf_exempt` are how access control is enforced at the function level.

The core idea in one sentence

> A decorator takes a function, **wraps it** with extra behaviour, and returns the enhanced version — without modifying the original function's code.

---

2. Prerequisites — Functions Are Objects

Decorators rely on one key insight: **in Python, functions are objects**. They can be assigned to variables, passed as arguments, and returned from other functions. This is called **first-class functions**.

If this seems strange, think of a function like a security tool — you can hand it to someone else to run, store it in a list of checks to execute later, or wrap it in a new tool that runs it and also logs the result.

In [ ]:
```
# Topic: functions are objects — they can be passed around like any other value

def check_port_open(port):
    """Returns True if the port is in the commonly-exposed list."""
    exposed_ports = {22, 80, 443, 8080, 3389}
    return port in exposed_ports


# Assign a function to a variable — no () means we're referencing it, not calling i
port_checker = check_port_open        # port_checker IS the function

print(type(check_port_open))          # <class 'function'>
print(port_checker(443))              # True  — calling via the alias
print(check_port_open is port_checker)  # True — same object, two names
```

In [ ]:
```
# Topic: passing a function as an argument to another function

def run_security_check(check_function, target):
    """Run any check function against a target and print the result."""
    result = check_function(target)   # call the function we received as an argumen
    print(f"Check '{check_function.__name__}' on {target}: {result}")
    return result


def is_internal_ip(ip):
    return ip.startswith("10.") or ip.startswith("192.168.")


def is_known_attacker(ip):
    blocklist = {"185.220.101.47", "45.33.32.156", "198.51.100.9"}
    return ip in blocklist


# Pass different functions to the same runner
run_security_check(is_internal_ip,     "10.0.0.14")
```

```
run_security_check(is_known_attacker,  "185.220.101.47")
run_security_check(is_known_attacker,  "10.0.0.14")
```

In [ ]:
```
# Topic: returning a function from another function — the foundation of decorators

def make_severity_checker(threshold):
    """Returns a function that checks if a CVSS score exceeds the threshold."""

    def check_severity(cvss_score):   # inner function — created fresh each time
        return cvss_score >= threshold

    return check_severity             # return the function — no () — not calling i


# Each call to make_severity_checker creates a different checker
is_critical = make_severity_checker(9.0)
is_high     = make_severity_checker(7.0)
is_medium   = make_severity_checker(4.0)

print(f"CVSS 9.8 is critical: {is_critical(9.8)}")   # True
print(f"CVSS 7.5 is critical: {is_critical(7.5)}")   # False
print(f"CVSS 7.5 is high:     {is_high(7.5)}")        # True
print(f"CVSS 3.1 is medium:   {is_medium(3.1)}")      # False
```

---

3. The Wrapper Pattern — Your First Decorator

Now that we know functions are objects, we can build a decorator.

A decorator is a function that:

1. Takes a function as its argument
2. Defines a new **inner (wrapper) function** that adds behaviour
3. Returns the inner function

```
def my_decorator(function):       # 1. takes a function as argument
    def wrapper(*args, **kwargs): # 2. defines a wrapper
        # ... do something before ...
        result = function(*args, **kwargs)  # call the original
        # ... do something after ...
        return result
    return wrapper                # 3. returns the wrapper (no
parentheses!)
```

The `@` syntax is shorthand for: `my_function = my_decorator(my_function)`

```
@my_decorator
def my_function():    # equivalent to: my_function =
my_decorator(my_function)
    pass
```

In [ ]:
```
# Topic: first real security decorator — audit logger
# Every decorated function automatically logs when it was called and what it return

import time

def audit_log(function):                    # 1. takes the function to decorate
    def wrapper(*args, **kwargs):           # 2. wrapper replaces the original
        timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
        print(f"[AUDIT] {timestamp} | CALL  | {function.__name__} | args={args}")

        result = function(*args, **kwargs)  # call the original function

        print(f"[AUDIT] {timestamp} | RETURN| {function.__name__} | result={result}
        return result                       # preserve the original return value

    return wrapper                          # 3. return the wrapper


# Apply the decorator
@audit_log
def check_cvss_severity(cvss_score):
    """Classify a CVSS score into a severity label."""
    if cvss_score >= 9.0:  return "CRITICAL"
    if cvss_score >= 7.0:  return "HIGH"
    if cvss_score >= 4.0:  return "MEDIUM"
    return "LOW"


@audit_log
def is_ip_blocked(ip_address):
    """Check if an IP is on the blocklist."""
    blocklist = {"185.220.101.47", "45.33.32.156", "198.51.100.9"}
    return ip_address in blocklist


# Now every call is automatically logged — no changes to the functions themselves
print("--- CVE severity checks ---")
check_cvss_severity(9.8)
check_cvss_severity(5.0)

print("\n--- IP blocklist checks ---")
is_ip_blocked("185.220.101.47")
is_ip_blocked("10.0.0.14")
```

How the `@` syntax works — step by step

```
@audit_log
def check_cvss_severity(cvss_score):
    ...
```

Python executes this exactly as if you had written:

```
def check_cvss_severity(cvss_score):
    ...
check_cvss_severity = audit_log(check_cvss_severity)
```

After decoration, `check_cvss_severity` is no longer the original function — it is the `wrapper` function returned by `audit_log`. When you call `check_cvss_severity(9.8)`, you are calling `wrapper(9.8)`, which logs, then calls the original, then logs again.

🎯 Practice — Basic Decorator

Exercise 1 — Write

Write a decorator `@validate_ip` that:

- Checks that the first positional argument to any decorated function is a valid IPv4 string (4 dot-separated parts, each 0-255)
- If valid: calls the original function and returns its result normally
- If invalid: prints `[ERROR] Invalid IP: <value>` and returns `None` without calling the original function

Apply it to a function `lookup_threat_intel(ip_address)` that just prints `Looking up: {ip_address}` and returns `True`.

Test with: `'10.0.0.14'` (valid), `'999.0.0.1'` (invalid), `'not_an_ip'` (invalid).

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

Exercise 2 — Predict

Before running the cell, predict the output line by line. Pay attention to: what runs at decoration time vs. what runs at call time.

In [ ]:
```
# Predict the output — then run to check!

def marker(function):
    print(f"Decorating: {function.__name__}")   # runs at DECORATION time
    def wrapper(*args, **kwargs):
        print(f"Calling: {function.__name__}")  # runs at CALL time
        return function(*args, **kwargs)
    return wrapper


print("-- defining functions --")

@marker
def scan_host(hostname):
    return f"scanned {hostname}"

@marker
def check_cve(cve_id):
    return f"checked {cve_id}"
```

```
print("-- calling functions --")
print(scan_host("web-01"))
print(check_cve("CVE-2021-44228"))
```

---

4. Generic Decorators with `*args` and `**kwargs`

In the previous section, the wrapper always used `*args, **kwargs` — not a fixed signature. This is essential: a decorator that only works with specific argument counts is barely reusable.

By using `*args, **kwargs` in the wrapper, the decorator becomes **universal** — it can wrap any function regardless of its signature, and passes all arguments through unchanged.

```
def my_decorator(function):
    def wrapper(*args, **kwargs):                # accepts ANY arguments
        result = function(*args, **kwargs)       # passes them ALL through
        return result
    return wrapper
```

This is the standard wrapper template. Always use it unless you have a specific reason to restrict the decorator to a particular signature.

In [ ]:
```
# Topic: universal timing decorator — works on any function

import time

def measure_time(function):
    """Decorator that measures and prints how long any function takes to run."""
    def wrapper(*args, **kwargs):
        start = time.time()
        result = function(*args, **kwargs)     # call original — any signature
        elapsed = time.time() - start
        print(f"[TIMING] {function.__name__} completed in {elapsed:.4f}s")
        return result
    return wrapper


@measure_time
def scan_port_range(host, start_port, end_port):
    """Simulate a port scan — 3 positional args."""
    time.sleep(0.1)    # simulate IO wait
    return [80, 443]   # pretend these were found open


@measure_time
def lookup_cve(cve_id, include_references=False):
    """Simulate a CVE lookup — 1 positional + 1 keyword arg."""
    time.sleep(0.05)
    return {"id": cve_id, "cvss": 9.8}
```

```
@measure_time
def get_blocklist():
    """No arguments — still works."""
    time.sleep(0.02)
    return ["185.220.101.47", "45.33.32.156"]


# Same decorator applied to three completely different function signatures
open_ports = scan_port_range("10.0.0.14", 1, 1024)
cve_data   = lookup_cve("CVE-2021-44228", include_references=True)
blocklist  = get_blocklist()

print(f"\nResults: {open_ports} | {cve_data['cvss']} | {len(blocklist)} IPs")
```

🎯 Practice — Generic Decorators

Exercise 3 — Write

Write a decorator `@log_exceptions` that:

- Wraps any function using `*args, **kwargs`
- Calls the original function inside a `try/except`
- If an exception occurs: prints `[EXCEPTION] {function_name}: {exception_type}: {message}` and returns `None`
- If no exception: returns the result normally

Apply it to:

- `parse_cvss(score_str)` — converts a string to float, raises `ValueError` on bad input
- `read_log_file(path)` — opens a file, raises `FileNotFoundError` if missing

Test with valid and invalid inputs for both functions.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

5. `functools.wraps` — Preserving Function Identity

When a decorator replaces a function with its wrapper, it **loses the original function's metadata**: name, docstring, parameter annotations, `__module__`, etc.

This is a real problem in security tools that use introspection to log function names, generate documentation, or inspect function signatures.

In [ ]:
```
# Topic: the metadata loss problem — without functools.wraps

def basic_decorator(function):
    def wrapper(*args, **kwargs):
        return function(*args, **kwargs)
    return wrapper


@basic_decorator
def check_firewall_rule(rule_id, action):
    """Validate a firewall rule. Returns True if the rule is well-formed."""
    return action in {"ALLOW", "DENY", "LOG"}


# After decoration, the function thinks its name is 'wrapper' — not 'check_firewall
print(f"__name__:    {check_firewall_rule.__name__}")
print(f"__doc__:     {check_firewall_rule.__doc__}")
print()
help(check_firewall_rule)   # shows 'wrapper' — misleading in logs and docs
```

In [ ]:
```
# Topic: functools.wraps — the fix — preserves all original metadata

from functools import wraps

def audit_log(function):
    @wraps(function)            # copies __name__, __doc__, __annotations__, etc.
    def wrapper(*args, **kwargs):
        print(f"[AUDIT] Calling {function.__name__}")
        result = function(*args, **kwargs)
        return result
    return wrapper


@audit_log
def check_firewall_rule(rule_id, action):
    """Validate a firewall rule. Returns True if the rule is well-formed."""
    return action in {"ALLOW", "DENY", "LOG"}


# Now the metadata is preserved correctly
print(f"__name__: {check_firewall_rule.__name__}")
print(f"__doc__:  {check_firewall_rule.__doc__}")
print()
help(check_firewall_rule)   # shows correct name and docstring
```

⚠️ Rule — Always Use `@wraps`

Every decorator you write should include `@wraps(function)` on the wrapper. The overhead is zero. The benefit — correct function names in logs, help text, and error tracebacks — is significant.

The standard decorator template with `@wraps`:

```
from functools import wraps

def my_decorator(function):
    @wraps(function)                          # always include this
    def wrapper(*args, **kwargs):
        # ... before ...
        result = function(*args, **kwargs)
        # ... after ...
        return result
    return wrapper
```

🎯 Practice — `functools.wraps`

Exercise 4 — Fix

The decorator below is missing `@wraps`. Fix it, then confirm that after applying the decorator, `intrusion_detector.__name__` is `'intrusion_detector'` and `intrusion_detector.__doc__` is not `None`.

In [ ]:
```
# FIX this decorator (missing @wraps):
# from functools import wraps   # <- uncomment and add @wraps

def rate_limit_check(function):
    # Missing @wraps here
    def wrapper(*args, **kwargs):
        return function(*args, **kwargs)
    return wrapper


@rate_limit_check
def intrusion_detector(source_ip, event_type):
    """Detect intrusion patterns from a source IP and event type."""
    suspicious_events = {"PORT_SCAN", "INTRUSION_ATTEMPT", "BRUTE_FORCE"}
    return event_type in suspicious_events


# These should print the correct values after your fix:
print(intrusion_detector.__name__)   # should be 'intrusion_detector'
print(intrusion_detector.__doc__)    # should be the docstring, not None
```

▶ 💡 **SOLUTION**

---

6. Parameterized Decorators

Sometimes you need to **configure** a decorator's behaviour when you apply it. For example: `@rate_limit(max_calls=5)` or `@require_role('admin')`.

A parameterized decorator is a **function that returns a decorator** — one extra level of nesting.

```
Plain decorator:         @my_decorator
Parameterized decorator: @my_decorator(param1, param2)
```

The structure adds one outer layer:

```
def my_decorator(param1, param2):    # outer: takes the config parameters
    def decorator(function):         # middle: the actual decorator
        @wraps(function)
        def wrapper(*args, **kwargs): # inner: runs on each call
            # use param1, param2 here
            return function(*args, **kwargs)
        return wrapper
    return decorator
```

Reading `@my_decorator(5)`: Python calls `my_decorator(5)` first, which returns a `decorator` function, and then that `decorator` is applied to the function below.

In [ ]:
```
# Topic: parameterized decorator — @require_min_severity(level)
# Only runs the function if the CVSS score meets a minimum severity

from functools import wraps

def require_min_severity(min_cvss):
    """Decorator factory — returns a decorator configured with min_cvss."""

    def decorator(function):         # actual decorator — takes the function
        @wraps(function)
        def wrapper(*args, **kwargs):
            # The first argument is expected to be the CVSS score
            cvss_score = args[0] if args else kwargs.get("cvss_score", 0)

            if cvss_score < min_cvss:
                print(f"[SKIP] {function.__name__}: CVSS {cvss_score} below thresho
                return None

            return function(*args, **kwargs)

        return wrapper
    return decorator


# Each function gets a different threshold
@require_min_severity(9.0)
def escalate_to_critical_team(cvss_score, cve_id):
    print(f"[CRITICAL ESCALATION] {cve_id} — CVSS {cvss_score} — notifying IR team"


@require_min_severity(7.0)
```

```python
def create_remediation_ticket(cvss_score, cve_id):
    print(f"[TICKET] {cve_id} — CVSS {cvss_score} — creating remediation task")


@require_min_severity(4.0)
def log_for_review(cvss_score, cve_id):
    print(f"[LOG] {cve_id} — CVSS {cvss_score} — added to review queue")


# Test with different CVSS scores
print("=== CVSS 9.8 (Critical) ===")
escalate_to_critical_team(9.8, "CVE-2021-44228")
create_remediation_ticket(9.8, "CVE-2021-44228")
log_for_review(9.8, "CVE-2021-44228")

print("\n=== CVSS 7.5 (High) ===")
escalate_to_critical_team(7.5, "CVE-2023-44487")  # skipped
create_remediation_ticket(7.5, "CVE-2023-44487")  # runs

print("\n=== CVSS 3.1 (Low) ===")
create_remediation_ticket(3.1, "CVE-2023-00001")  # skipped
log_for_review(3.1, "CVE-2023-00001")             # skipped
```

In [ ]:

```python
# Topic: parameterized decorator — @retry(max_attempts, delay)
# Retries a function if it raises an exception — common for network operations

from functools import wraps
import time

def retry(max_attempts=3, delay_seconds=0.1):
    """Retry a function up to max_attempts times if it raises an exception."""

    def decorator(function):
        @wraps(function)
        def wrapper(*args, **kwargs):
            last_exception = None

            for attempt in range(1, max_attempts + 1):
                try:
                    return function(*args, **kwargs)
                except Exception as e:
                    last_exception = e
                    print(f"[RETRY] {function.__name__} attempt {attempt}/{max_atte
                    if attempt < max_attempts:
                        time.sleep(delay_seconds)

            # All attempts exhausted
            print(f"[FAILED] {function.__name__} failed after {max_attempts} attemp
            raise last_exception

        return wrapper
    return decorator


# Simulates a flaky API call that fails twice then succeeds
call_count = 0
```

```
@retry(max_attempts=3, delay_seconds=0.0)
def query_threat_intel_api(ip_address):
    global call_count
    call_count += 1
    if call_count < 3:                      # fail first 2 attempts
        raise ConnectionError("API timeout")
    return {"ip": ip_address, "threat_score": 92}


try:
    result = query_threat_intel_api("185.220.101.47")
    print(f"[SUCCESS] Result: {result}")
except Exception as e:
    print(f"[ERROR] Final failure: {e}")
```

🎯 Practice — Parameterized Decorators

Exercise 5 — Write

Write a parameterized decorator `@rate_limit(max_calls)` that:

- Tracks how many times the decorated function has been called (use a closure variable)
- Allows up to `max_calls` calls to succeed normally
- On the `max_calls + 1` th call and beyond: prints `[RATE LIMIT] {function_name} exceeded limit of {max_calls} calls` and returns `None` without calling the original

Apply it: `@rate_limit(max_calls=3)` on `submit_alert(alert_id, severity)` which just prints `Submitted: {alert_id} [{severity}]`.

Test by calling it 5 times — first 3 should succeed, last 2 should be blocked.

In [ ]:
```
# Your code here
from functools import wraps
```

▶ 💡 **SOLUTION**

---

7. Stacking Multiple Decorators

You can apply multiple decorators to one function by stacking them. They apply **bottom-up** — the decorator closest to `def` applies first.

```
@decorator_A
@decorator_B
@decorator_C
def my_function():
    pass
```

```
# Equivalent to:
my_function = decorator_A(decorator_B(decorator_C(my_function)))

# Execution order when called:
# decorator_A wrapper runs first (outermost)
# decorator_B wrapper runs second
# decorator_C wrapper runs third (innermost — closest to the real
function)
# original function runs
# decorator_C wrapper finishes
# decorator_B wrapper finishes
# decorator_A wrapper finishes
```

Think of it like layers of security: the outermost layer (authentication) runs first, then authorisation, then rate limiting, then the actual function.

In [ ]:

```
# Topic: stacking decorators — building a layered security pipeline

from functools import wraps
import time

# --- Decorator 1: audit logging ---
def audit_log(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        print(f"[AUDIT] {function.__name__} called")
        result = function(*args, **kwargs)
        print(f"[AUDIT] {function.__name__} returned: {result}")
        return result
    return wrapper


# --- Decorator 2: timing ---
def measure_time(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = function(*args, **kwargs)
        print(f"[TIMING] {function.__name__}: {(time.time() - start)*1000:.1f}ms")
        return result
    return wrapper


# --- Decorator 3: exception logging ---
def log_exceptions(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        try:
            return function(*args, **kwargs)
        except Exception as e:
            print(f"[EXCEPTION] {function.__name__}: {type(e).__name__}: {e}")
            return None
    return wrapper
```

```
# Stack all three — execution order: audit_log -> measure_time -> log_exceptions ->
@audit_log
@measure_time
@log_exceptions
def assess_vulnerability(cve_id, cvss_score):
    """Assess a vulnerability — raises if CVSS is out of range."""
    if not (0.0 <= cvss_score <= 10.0):
        raise ValueError(f"Invalid CVSS score: {cvss_score}")
    severity = "CRITICAL" if cvss_score >= 9.0 else "HIGH" if cvss_score >= 7.0 els
    return {"cve": cve_id, "severity": severity}


print("=== Valid input ===")
assess_vulnerability("CVE-2021-44228", 10.0)

print("\n=== Invalid input (exception handled by log_exceptions) ===")
assess_vulnerability("CVE-BAD", 15.0)
```

🎯 Practice — Stacking

Exercise 6 — Write

Apply two decorators to a single function:

- `@measure_time` (from above, or rewrite it)
- `@validate_ip` (from Exercise 1, or rewrite it)

Stack them on `geolocate_ip(ip_address)` which just prints `Geolocating: {ip_address}` and returns `'Belgium'`.

Test with a valid IP and an invalid IP. Predict: which decorator runs first? Which decorator should be outermost for the most sensible behaviour?

In [ ]:
```
# Your code here
from functools import wraps
import time
```

▶ 💡 **SOLUTION**

---

8. Real Security Decorator Patterns

These are the four decorator patterns you will encounter and build most often in real security work:

Pattern	What it does	When to use
Audit log	Records every call to a function with timestamp and args	Security-sensitive functions
Rate limiter	Blocks a function after N calls in a window	API calls, alert submissions
Input validator	Rejects calls with bad arguments before they reach the function	Any function processing untrusted input
Cache (memoize)	Returns stored result if same args seen before	Expensive lookups (VirusTotal, CVE DB)

In [ ]:

```
# Topic: memoize/cache decorator — avoid redundant threat intel lookups
# Real tools use this for VirusTotal, Shodan, or internal CVE database queries

from functools import wraps

def cache_result(function):
    """Cache function results — same arguments return the cached result immediately
    _cache = {}   # dict stored in closure — persists between calls

    @wraps(function)
    def wrapper(*args, **kwargs):
        # Build a hashable cache key from all arguments
        cache_key = (args, tuple(sorted(kwargs.items())))

        if cache_key in _cache:
            print(f"[CACHE HIT]  {function.__name__}{args} — returning cached resul
            return _cache[cache_key]

        print(f"[CACHE MISS] {function.__name__}{args} — calling API")
        result = function(*args, **kwargs)
        _cache[cache_key] = result
        return result

    return wrapper


@cache_result
def query_virustotal(ip_address):
    """Look up an IP on VirusTotal — expensive API call (simulated)."""
    # In a real tool: requests.get('https://www.virustotal.com/api/v3/ip_addresses/
    return {"ip": ip_address, "malicious": ip_address.startswith("185."), "score":


# First call: hits the API
result1 = query_virustotal("185.220.101.47")
# Second call with same IP: returns cached result instantly — no API call
result2 = query_virustotal("185.220.101.47")
# Different IP: cache miss — calls API again
result3 = query_virustotal("10.0.0.14")
# Same IP as line 1: cache hit
result4 = query_virustotal("185.220.101.47")
```

```
print(f"\nResult 1 == Result 2: {result1 == result2}")
```

🎯 Practice — Real Patterns

Exercise 7 — Write

Write a parameterized decorator `@require_role(allowed_roles)` that:

- Takes a set or list of allowed role strings, e.g. `{'admin', 'soc_analyst'}`
- The decorated function must accept `role` as its first keyword argument
- If `role` is in `allowed_roles`: call the function normally
- Otherwise: print `[DENIED] Role '{role}' not authorised for {function_name}` and return `None`

Apply it:

- `@require_role({'admin'})` on `delete_firewall_rule(rule_id, role)`
- `@require_role({'admin', 'soc_analyst'})` on `view_alert(alert_id, role)`

Test with roles: `'admin'`, `'soc_analyst'`, `'intern'`.

In [ ]:
```
# Your code here
from functools import wraps
```

▶ 💡 **SOLUTION**

---

9. Summary and Next Steps

Key Rules

Rule	Why it matters
Always use `@wraps(function)`	Preserves `__name__`, `__doc__` — correct in logs and tracebacks
Always use `*args, **kwargs` in wrapper	Makes the decorator work on any function signature
Always `return result` from the wrapper	Preserves the original function's return value
Parameterized = 3 levels of nesting	outer(config) → decorator(fn) → wrapper(args)
Stack decorators bottom-up	Closest to `def` applies first
Order matters	Validate input before timing; log before rate-limiting

The standard template — memorise this

```
from functools import wraps

def my_decorator(function):           # plain decorator
    @wraps(function)
    def wrapper(*args, **kwargs):
        result = function(*args, **kwargs)
        return result
    return wrapper


def my_param_decorator(param):        # parameterized decorator
    def decorator(function):
        @wraps(function)
        def wrapper(*args, **kwargs):
            result = function(*args, **kwargs)
            return result
        return wrapper
    return decorator
```

What comes next

- **01_Drills_Decorators.ipynb** — 15 exercises